Dayli AI — Daily Living Solutions Engine

An AI-powered platform that helps people with disabilities, caregivers, and families discover personalized adaptive solutions for daily living challenges.

Project Summary & Technical Documentation — April 2026

---

1. Project Overview

Dayli AI is a Next.js web application that combines conversational AI assessment with keyword-based search to match users with adaptive tools, strategies, and products for Activities of Daily Living (ADLs).

Core Value Proposition

- **Empathetic AI Assessment:** Socratic-style conversational interview powered by Claude that understands the user's unique challenges
- **Smart Solution Matching:** Keyword-based search with category and tag scoring across a curated solutions catalog, with semantic search infrastructure ready for future activation
- **Community Trust:** Ratings, reviews, and feedback from other users to validate recommendations
- **Personalized Dashboard:** Pattern insights, liked solutions, and custom collections
- **Accessibility-First:** Built-in themes (light, dark, high contrast, grayscale) and large text options

---

2. Technology Stack

Frontend

Technology	Purpose
Next.js 16.2	React 19, App Router, Turbopack
Tailwind CSS 4	Utility-first styling
Lucide React	Icon library

TypeScript 5	Strict mode

Backend & Database

Technology	Purpose
Supabase	PostgreSQL + Auth + RLS + pgvector
Next.js API Routes	Server-side endpoints
Row Level Security	Per-user data isolation

AI & Machine Learning

Technology	Purpose	Status
Claude Opus 4.6	Socratic assessment chat	Active
Claude Sonnet 4	Pattern insight generation	Active
OpenAI text-embedding-3-small	1536-dim vector embeddings for semantic search	Not yet configured (API key placeholder)

Infrastructure

Technology	Purpose
Vercel	Hosting & deployment
Supabase Cloud	Managed database & auth
Web Speech API	Voice input (desktop)

Key Dependencies

Package	Version	Purpose
next	16.2.1	React framework with App Router
react / react-dom	19.2.4	UI rendering
@anthropic-ai/sdk	^0.82.0	Claude API integration
openai	^6.33.0	Embedding generation (not yet active)

@supabase/supabase-js	^2.101.1	Database & auth client
@supabase/ssr	^0.10.0	Server-side auth with cookies
lucide-react	^1.7.0	Icon components
tailwindcss	^4	CSS framework

---

3. Application Pages & User Flows

Route	Page	Description	Auth
`/`	Landing Page	Hero section, animated previews, trust signals, Daily Living Labs partnership, contact form	PUBLIC
`/diagnostic`	Diagnostic Tool	4-step questionnaire: user profile, care type, disability category, specific challenges with keyword tagging	PUBLIC
`/assessment`	AI Chat Assessment	Socratic interview with Claude, streaming responses, voice input, session history sidebar	PROTECTED
`/solutions`	Solution Results	Paginated grid with filtering (source, category, DIY, price), sorting, feedback, and like/save	PUBLIC
`/dashboard`	User Dashboard	Personalized greeting, AI chat input, suggestion pills, pattern insights, liked solutions, collections	PROTECTED
`/collections`	Collections Manager	View all collections, liked items, individual collection contents; create, edit, delete collections	PROTECTED

`/profile`	Profile & Settings	Avatar upload, username/email/password management, theme & accessibility settings, account deletion	PROTECTED
`/request-form`	Solution Request	Custom solution request form when no matches found; pre-fills diagnostic data	PUBLIC
`/request-success`	Request Confirmation	Confirmation page after solution request submission	PUBLIC
`/daily-living-labs`	DLL Partnership	Informational page about Daily Living Labs collaboration	PUBLIC
`/auth/sign-in`	Authentication	Modal with Google/Facebook/Apple OAuth + email/password sign-in/sign-up	PUBLIC
`/no-results`	No Results	Shown when zero solutions match the diagnostic assessment	PUBLIC

Primary User Flow

1. **Landing Page** — User arrives and chooses Diagnostic Tool or AI Chat
2. **Assessment** — Either multi-step questionnaire (Diagnostic) or conversational AI interview (Chat)
3. **Solution Matching** — Categories & keywords extracted; keyword matching runs against the `all_solutions` view
4. **Results** — Filtered, sorted solutions displayed with relevance scores, images, and metadata
5. **Engagement** — User likes, saves to collections, rates, provides feedback on solutions
6. **Dashboard** — Personalized view with pattern insights, history, and quick access to saved solutions

---

4. API Architecture

All API endpoints are implemented as Next.js App Router API routes. They use Supabase for data persistence and integrate with Claude for AI-powered features.

Endpoint	Method	Purpose	AI Model
`/api/assessment`	POST	Socratic needs assessment conversation; extracts categories, keywords, ADL focus	Claude Opus 4.6
`/api/assessment/sessions`	GET	Fetch user's recent assessment sessions for history sidebar	—
`/api/match`	POST	Solution matching: keyword scoring via `match_solutions()` RPC with 3-tier fallback	— (semantic search ready but inactive)
`/api/solutions`	GET	Batch fetch solution details by IDs	—
`/api/embed`	POST	Generate & store solution embeddings (maintenance; requires bearer token)	OpenAI Embed (inactive)
`/api/likes`	GET / POST	Get liked solution IDs; toggle like/unlike	—
`/api/collections`	GET / POST	CRUD for collections; add/remove solution items	—
`/api/feedback`	POST	Record helpful/not-helpful feedback for solutions	—
`/api/feedback/summary`	GET	Aggregate helpful and not-helpful solution IDs for a user	—

`/api/patterns`	GET	Analyze assessment history and generate personalized insights	Claude Sonnet 4
`/api/ratings`	GET / POST	Community ratings (1-5 stars) with optional review text	—
`/api/solution-request`	POST	Store custom solution requests when no matches found	—

Data Architecture

```
Browser (React 19)
    |
    |--- Client Supabase (anon key, RLS enforced)
    |         |
    |         +---> Supabase Auth (sessions, OAuth)
    |         +---> Direct queries (with user context)
    |
    +--- Next.js API Routes (server-side)
            |
            +--- Supabase Admin (service role, bypasses RLS)
            |         +---> PostgreSQL (all tables)
            |         +---> pgvector (solution_embeddings) [infrastructure re
            |         +---> RPC functions (match_solutions, match_solutions_s
            |
            +--- Claude API (Anthropic SDK)
                      +---> Opus 4.6: Socratic assessment
                      +---> Sonnet 4: Pattern insights
```

Note: OpenAI embeddings infrastructure (lib/embeddings.ts, /api/embed, match_solutions_semantic) is fully coded and ready but not yet active — the `OPENAI_API_KEY` is currently a placeholder. The system gracefully falls back to keyword-based matching via `match_solutions()`.

---

5. Database Schema

The database uses Supabase (PostgreSQL + pgvector extension). All tables have Row Level Security (RLS) enabled.

Core Tables

Table	Key Columns	Purpose
profiles	id (FK → auth.users), display_name, avatar_url, role	User profile data, auto-created on signup via trigger
solutions	id, title, description, adl_category, disability_tags[], price_tier, is_diy, source_url, cover_image_url, what_made_it_work, is_active	Core catalog of assistive solutions (renamed from internal_solutions)
solution_embeddings	solution_id, embedding (pgvector 1536-dim), chunk_text	Vector embeddings for semantic search (infrastructure ready, not yet populated)
community_ratings	user_id, solution_id, rating (1-5), review_text	Star ratings & optional reviews (one per user per solution)
solution_likes	user_id, solution_id	User bookmarks / favorites
collections	id, user_id, name, color	User-created solution groups (8 color options)
collection_items	collection_id, solution_id	Many-to-many junction: collections <-> solutions
solution_pathways	user_id, session_id, title, solutions (jsonb), status	User's solution journey with step tracking
assessment_sessions	session_id, user_id, messages (jsonb), extracted_categories[],	Full AI chat history per assessment

	extracted_keywords[], adl_focus, role, completed	
needs_assessments	user_id, session_id, role, categories[], keywords[], adl_focus, severity_hints	Structured assessment output for matching
challenge_categories	slug, label, description, icon_emoji, sort_order	8 seeded ADL categories (reference table)
solution_feedback	user_id, solution_id, session_id, is_helpful	Thumbs up/down feedback signals
solution_requests	user_id, email, description, budget, diagnostic_profile, status	Custom solution requests
community_submissions	id, title, description, adl_category, tags[], pricing, photos[], website_url, status	User-submitted solutions awaiting moderation

Diagnostic Tree Tables (Legacy — scaffolded, not yet operational)

Table	Purpose
diagnostic_sessions	Tracks user's path through the decision tree
diagnostic_prompts	Defines question nodes with options (no seed data yet)
diagnostic_paths	Edges connecting prompts based on selected options (no seed data yet)
diagnostic_results	Matched criteria output from completed sessions

Search & Filter Tables

Table	Purpose
solution_results	Cached search results (internal + external) with relevance scores
solution_tags	Junction table applying filter tags to results
filter_options	Reference table for filter groups (price, source_type, adl_category)

Views

`all_solutions` — Unified view combining:

- Curated solutions from the `solutions` table (where `is_active = true`)
- Approved community submissions from `community_submissions` (where `status = 'approved'`)

Both `match_solutions` functions query this view, not the `solutions` table directly.

Supabase RPC Functions

Function	Type	Description
match_solutions()	Keyword-based	Scores by ADL category match (1.0 weight) + disability tag overlap (0.5 weight). Currently the primary matching method.
match_solutions_semantic()	Vector similarity	Combines pgvector cosine similarity (70% weight) + category match (30% weight). Ready but inactive until embeddings are generated.

Matching Strategy (3-tier fallback)

1. **Keyword Match** — Primary method. Uses `match_solutions()` with extracted categories and keywords from the assessment
2. **Full Catalog** — If keyword match returns zero results, fetches all active solutions as a last resort
3. **Semantic Match** — Infrastructure ready. When OpenAI API key is configured and embeddings are generated, `match_solutions_semantic()` will be tried first before keyword fallback

Seeded Data

- **8 ADL categories:** bathing, dressing, eating, mobility, toileting, transferring, communication, memory
- **24 sample solutions** across all ADL categories with disability tags, pricing, DIY flags, and testimonials

---

6. AI Integration Details

Claude — Socratic Assessment (`/api/assessment`)

Aspect	Detail
Model	`claude-opus-4-6`
Max tokens	500
Purpose	Multi-turn empathetic interview (3-5 questions) about ADL challenges

1. **System Prompt** — 47-line instruction set for warm, empathetic interviewing style. Asks 3–5 focused questions about ADL challenges.
2. **Multi-turn Dialogue** — Claude maintains conversation context, adapts questions based on user responses, and generates suggestion pills for quick replies.
3. **Structured Extraction** — When assessment is complete, Claude outputs `ASSESSMENT_COMPLETE` JSON with categories, keywords, ADL focus, and user role.
4. **Session Persistence** — Full conversation stored in `assessment_sessions`; extracted data saved to `needs_assessments`.

Claude — Pattern Insights (`/api/patterns`)

Aspect	Detail
Model	`claude-sonnet-4-20250514`
Max tokens	200
Purpose	Analyze user's assessment history and generate 2-3 sentence personalized insights

- Analyzes up to 20 recent assessments per user
- Counts ADL category frequencies
- Considers feedback stats (helpful vs unhelpful)
- Generates a short, warm summary displayed on the dashboard's "Your Insights" card

- Uses Sonnet (faster, cheaper) instead of Opus since this is a simple summarization task

OpenAI — Semantic Search (Not Yet Active)

Aspect	Detail
Model	`text-embedding-3-small` (1536 dimensions)
Status	Code complete, API key not yet configured
Location	`lib/embeddings.ts`, `/api/embed`

The embedding infrastructure is fully built:

- `generateEmbedding()` — generates 1536-dim vectors from text
- `embedSolution()` — generates and stores embedding for a solution
- `/api/embed` — maintenance endpoint to batch-embed all solutions
- `/api/match` — checks for embeddings first, falls back to keywords

To activate: Set a valid `OPENAI_API_KEY` in `.env.local` and run the `/api/embed` endpoint to generate embeddings for all solutions.

---

7. Component Architecture

Component	Description	Used In
AuthButton	Dropdown with sign-in/sign-out, dashboard link, profile link; shows avatar when authenticated	All page headers
SolutionCard	Solution preview with image, relevance badge, like/share/save/feedback buttons, rating display	Solutions, Dashboard, Collections
SolutionModal	Full-detail modal with image, tags, description, source link, price; keyboard & click-outside dismiss	Solutions, Dashboard, Collections
SolutionMap	Visual mapping/organization of solutions	Solutions

ShareModal	Social sharing options and copy-to-clipboard functionality	SolutionCard
CollectionTooltip	Dropdown for saving solutions to color-coded collections	SolutionCard

Custom React Hooks

Hook	Purpose
useUser()	Client-side auth state management. Returns user object, loading state, signOut function. Listens to Supabase auth state changes in real-time.
useLikesAndCollections()	Manages likes & collections state. Returns likedIds, collections, toggle/CRUD functions. Optimistic UI updates with error reversion.

---

8. Authentication System

Supported Methods

- **OAuth Providers:** Google, Facebook, Apple (via Supabase OAuth) — requires configuration in each provider's developer console and Supabase Dashboard
- **Email/Password:** Sign up with display name; sign in with credentials

Auth Flow

1. **Sign In/Up** — Modal UI at `/auth/sign-in` with social buttons and email form
2. **OAuth Callback** — `/auth/callback` exchanges code for session, redirects to target page
3. **Session Management** — Supabase SSR with cookie-based sessions via `@supabase/ssr`
4. **Protected Routes** — `useUser()` hook redirects unauthenticated users to sign-in with `?next=` return URL
5. **Profile Auto-creation** — Database trigger creates a `profiles` row on first signup, extracting display name from metadata or email prefix

OAuth Provider Status

Provider	Code Ready	Console Configured
Google	Yes	Pending — needs OAuth client in Google Cloud Console
Facebook	Yes	In progress — Meta Developer account created
Apple	Yes	Pending — needs Apple Developer Program membership

---

9. Accessibility & Theming

Feature	Implementation	Storage
Light Theme	Default appearance	localStorage: `dayli-theme`
Dark Theme	Dark backgrounds, light text	
High Contrast	Maximum contrast ratios for low-vision users	
Grayscale	Color-blind friendly monochrome mode	
Large Text	Increases base font size across application	localStorage: `dayli-large-text`

---

10. Diagnostic Tool

A multi-step guided questionnaire that helps users who prefer structured input over free-form conversation. The diagnostic tool UI is implemented in `/diagnostic` but the backend decision tree tables (`diagnostic_prompts` , `diagnostic_paths`) do not yet have seed data in migrations.

1. **User Profile** — "Are you looking for solutions for yourself or someone else?"
2. **Care Context** — Type of care received: Therapy, Home Care, or None
3. **Disability Category** — Select from categories: Mobility, Dexterity, Vision, Hearing, Cognitive
4. **Specific Challenges** — Detailed challenge selection within chosen category; maps to ADL keywords

After completion, categories and keywords are passed to `/solutions` page for matching.

---

11. User Engagement Features

Feature	Description	Storage
Likes / Bookmarks	Toggle heart icon on any solution card. Accessible from Dashboard and Collections page.	`solution_likes` table
Collections	Create unlimited color-coded collections (8 colors). Add/remove solutions; rename or delete collections.	`collections` + `collection_items`
Feedback	Thumbs up/down on each solution. Used to improve future recommendations. "Exclude Not Helpful" filter on results page.	`solution_feedback` table
Community Ratings	1-5 star ratings with optional text reviews. Aggregated average shown on solution cards. Sortable by "Most Helpful" on results page.	`community_ratings` table
Pattern Insights	AI-generated summary of user's assessment history. Shows category frequency analysis on dashboard. Powered by Claude Sonnet 4.	Generated on-demand via `/api/patterns`
Solution Requests	Submit custom requests when no matches found. Includes budget preference and contact email. Pre-fills with diagnostic profile data.	`solution_requests` table

---

12. ADL Categories & Solution Taxonomy

ADL Categories (8 seeded)

Slug	Label	Icon
bathing	Bathing	🛁
dressing	Dressing	👕
eating	Eating	🍽️
mobility	Mobility	♿
toileting	Toileting	🚽
transferring	Transferring	🔄
communication	Communication	💬
memory	Memory	🧠

Solution Attributes

- **Price Tier:** Free, $ (Low), $$ (Mid), $$$ (High)
- **Type:** DIY or Pre-Made
- **Source:** Web, YouTube, Community
- **Disability Tags:** wheelchair, arthritis, one-handed, stroke, tremors, parkinsons, balance, etc.
- **Relevance Score:** 0–1 (displayed as 1–99%)

---

13. Environment Configuration

Variable	Purpose	Required	Status
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL (public, used client-side)	Yes	Configured
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anonymous key (public, respects RLS)	Yes	Configured

`SUPABASE_SERVICE_ROLE_KEY`	Supabase admin key (server-only, bypasses RLS)	Yes	Configured
`DAYLI_ANTHROPIC_KEY`	Anthropic API key for Claude models	Yes	Configured
`OPENAI_API_KEY`	OpenAI API key for embeddings	Yes (for semantic search)	Placeholder — not yet active
`FIRECRAWL_API_KEY`	Firecrawl web scraping API key	Optional	Placeholder
`SUPADATA_API_KEY`	Supadata API key	Optional	Placeholder
`EMBED_API_SECRET`	Bearer token for embedding maintenance endpoint	Optional	Not set

Deployment: The application is deployed on Vercel at dayli-ai.vercel.app with automatic deployments from the main branch. Environment variables are configured in the Vercel dashboard.

---

14. Migration History

Migration	Purpose
`20260401000001`	Profiles table with auto-creation trigger
`20260402000002`	Likes & collections tables with RLS
`20260403000003`	Core tables: solutions, embeddings, ratings, assessments, categories, pathways; match functions; 24 seeded solutions
`20260404000004`	Enable RLS on all remaining tables

`20260404000005`	Drop unused chat tables
`20260405000006`	Add user_id to solution_feedback
`20260405000007`	Add user_id to assessment_sessions
`20260405000008`	Create all_solutions view combining curated + community submissions

---

Generated for the Dayli AI project — April 2026